

第9章 并发与异常处理

建议学时:5-7 小时 | **难度:**★★★★★ | **读者背景:**熟悉 C 的 `errno` 错误处理与 `pthread` 线程(或至少了解线程概念),听说过数据竞争

本章学习目标

- 掌握错误处理三件套的分工:可恢复错误用 `Result`,不可恢复用 `panic!`, "快速传播"用 `?`(对照 `errno` 与 `abort`)
- 掌握 `std::thread` 的创建、`join` 与 `move` 闭包,对照 `pthread_create`/`pthread_join`
- 理解 `Send`/`Sync` 两个标记 `trait`: 数据竞争从"运行期灾难"变成"编译期拒绝"
- 掌握 `Arc<Mutex<T>>` 组合:共享可变状态的唯一正道;理解锁中毒
- 掌握 `RwLock` 与原子类型,对照 C11 的 `atomic` 与读写锁
- 掌握 `mpsc` 通道:多生产者单消费者模型,对照 `pipe` 与消息队列
- 理解死锁的成因与预防:锁顺序、持锁作用域、无锁优先
- 掌握 `tokio` 异步入门:`async`/`await` 的直觉、与线程的取舍、何时用

从 C 到 Rust:本章主题如何迁移

这一章是两个世界的正面碰撞。错误处理:C 用 `errno`(全局变量,可被任意函数覆盖)、返回值(`-1 + errno`)、`abort()`。Rust 用三种显式形态:`Result`(可恢复,值是数据)、`panic!`(不可恢复,栈展开或 `abort`)、`?`(传播,一行代替三行样板)。关键不是"哪种更好",而是"类型系统把错误分成了几类,每类各有出口"——C 里所有错误都是"负返回值 + `errno`",于是经常忘了查。

并发:C 的 `pthread` 给你一把刀,也给你全部伤害自己的能力:共享变量不加锁是数据竞争(未定义行为)、加错锁是死锁、忘记 `join` 是悬垂。Rust 的态度是"能静态保证的,绝不留到运行期":所有权(第5章)保证了每个值只有一个所有者,于是——数据要么单线程独享(`move` 进线程),要么多线程共享(必须 `Send`/`Sync` 且经互斥/原子访问)。`Send`/`Sync` 是两个自动推导的标记 `trait`:你的类型天然声明"我能/不能安全地跨线程传递"。写错并发代码不再是"偶尔崩溃的玄学",而是"编译器拒绝的必然"。本章从头到尾,你要带着一个对照表看:C 的每一次并发冒险,在 Rust 里对应哪个类型、哪条规则。

前置知识自查

- C 的 `errno`、`perror`、`strerror`;函数返回 `-1` 表示失败的惯例;`abort()` 与 `exit()`
- `pthread` 的基本使用:`pthread_create`/`join`/`mutex_lock`/`unlock`(至少见过)
- 第5章的所有权与 `move` 语义——本章线程的 `move` 闭包直接依赖它
- 第8章的每连接一线程模式(已用过 `thread::spawn`)
- (可选)C11 的 `atomic` 头文件、生产者-消费者模型的直觉

本章学习路线

1. **第一阶段(约1.5小时):**§9.1–§9.2,错误处理三件套与线程——先跑通"多线程 + Result"
2. **第二阶段(约2小时):**§9.3–§9.4,Send/Sync、Mutex 与原子——本章最难也最值钱的部分,反复对照 C 的数据竞争
3. **第三阶段(约1.5小时):**§9.5–§9.6,通道与死锁——生产者-消费者模型落地
4. **第四阶段(约1小时):**§9.7,tokio 异步入门——知道"什么时候该换武器"
5. **验收标准:**能徒手写一个"多生产者多消费者 + 共享计数"的程序,并说出每个 Arc/Mutex/通道选择的原因

本章知识导图

- §9.1 错误处理三件套:Result / ? / panic!(vs errno)
- §9.2 线程:std::thread 与 move 闭包(vs pthread)
- §9.3 Send 与 Sync:数据竞争的编译期防线
- §9.4 共享可变状态:Arc<Mutex<T>>、RwLock 与原子
- §9.5 通道:mpsc 生产者-消费者
- §9.6 死锁:成因、识别与预防
- §9.7 tokio 异步入门:async/await
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§9.1 错误处理三件套:Result / ? / panic!(vs errno)

9.1.1 C 的 errno 之痛

```
// C:错误与返回值混在一起,errno 是全局状态
#include <errno.h>
#include <string.h>

int read_config(const char *path, char *buf, size_t n) {
    FILE *f = fopen(path, "r");
    if (f == NULL) return -1;           // 原因在 errno 里
    if (fread(buf, 1, n, f) != n) {    // 读不完整
        fclose(f);
        return -1;                     // errno 可能已被内部调用覆盖!
    }
    fclose(f);
    return 0;
}

// 调用方:if (read_config(...) == -1) { perror("config"); }
// 问题:多线程下 errno 是线程局部的还好,但"谁覆盖了 errno"永远说不清
```

errno 的三宗罪:错误与返回值分离(要查两处)、全局状态(线程内可被任何库函数改写)、错误码没有携带上下文(不知道是哪个文件、哪个操作)。

9.1.2 Rust 的三件套

```
use std::fs;

/// 可恢复错误:返回 Result。错误携带上下文(路径 + 原因),随值传递
fn read_config(path: &str) -> std::io::Result<String> {
    let text = fs::read_to_string(path)?;    // ? :失败就带着错误提前返回
    if text.trim().is_empty() {
        // 业务规则错误:也是可恢复的,用自定义错误表达
        return Err(std::io::Error::new(
            std::io::ErrorKind::InvalidData,
            format!("配置文件 {path} 为空")));
    }
    Ok(text)
}

/// 不可恢复错误:panic!。用于"前置条件被破坏,程序无法继续"
fn main() {
    let cfg = match read_config("app.toml") {
        Ok(text) => text,
        Err(e) => {
            // 顶层兜底:打印完整错误并优雅退出(对照 perror + exit(1))
            eprintln!("启动失败:{e}");
            std::process::exit(1);
        }
    };

    // 断言式 panic:不可达条件。release 构建仍检查(对照 assert(3))
    assert!(cfg.contains("server"), "配置文件缺少 server 段");
    println!("配置加载成功");
}
```

C	Rust	语义
返回 -1 + errno	Result<T, E>:Ok(T) / Err(E)	错误是值,带上下文,随调用链传递
perror / strerror	错误 Display 打印(含路径/原因)	信息完整,不猜
逐层 if (ret < 0) return -1	?	一行完成"失败即返回"
abort()	panic!(默认栈展开,可配 abort)	不可恢复;但 panic 前可运行 Drop 清理
assert(cond) 宏	assert!(cond) / assert_eq!	不变式检查,失败即 panic
全局错误码(errno)	错误随值传递	无全局状态,线程/库调用互不污染

⚠ 易错点 1:unwrap/expect 是"开发期 OK,生产期雷"

unwrap() 在 Err 时直接 panic。开发与原型随便用;生产代码在"错误可预期"的路径上(文件缺失、网络失败、解析失败)一律用 ? 或 match, unwrap 只留给"逻辑上不可能失败"的调用(如写死的字面量解析)。经验法则:release 构建里每个 unwrap 都是一个潜在的 panic 点,代码审查时逐一点名。

§9.2 线程:std::thread 与 move 闭包

```
use std::thread;
use std::time::Duration;

fn main() {
    // 对照 pthread_create:spawn 接收闭包(线程体),返回句柄
    let handle = thread::spawn(|| {
        for i in 1..=3 {
            println!("工作线程:{i}");
            thread::sleep(Duration::from_millis(200));
        }
    });

    // 主线程继续干活
    for i in 1..=3 {
        println!("主线程:{i}");
        thread::sleep(Duration::from_millis(150));
    }

    // 对照 pthread_join:等待工作线程结束;返回值是 Result
    handle.join().expect("工作线程 panic 了");

    // 带返回值的线程:闭包的最后一个表达式就是线程的"退出码"
    let calc = thread::spawn(|| 6 * 7);
    let answer = calc.join().expect("计算线程崩溃");
    println!("计算结果:{answer}");
}
```

9.2.1 move:把数据送进线程

```
use std::thread;

fn main() {
    let data = String::from("这是要给线程的数据");

    // 不 move:闭包借用 data,而 data 属于主线程—生命周期冲突,编译错误
    // let t = thread::spawn(|| println!("{data}")); // ERROR: may outlive...

    // move:把 data 的所有权转移进新线程(第5章的转移语义)
    let t = thread::spawn(move || {
        println!("线程收到:{data}"); // data 归线程所有,主线程不能再碰
    });
    // println!("{data}"); // ERROR: value borrowed after move

    t.join().unwrap();
}
```

C 的"传参给线程"靠 malloc + 指针 + 约定"谁 free";Rust 的 move 闭包把所有权转移显式化,编译器保证:要么数据在线程里,要么在创建者手里,不会两边同时悬垂。这就是"数据竞争预防"的第一道闸门。

§9.3 Send 与 Sync:数据竞争的编译期防线

9.3.1 两个标记 trait 的定义

Send:类型可以安全地转移所有权到另一个线程(TcpStream、String、Box 都是 Send)。**Sync**:类型可以安全地被多个线程同时借用(&T 可以跨线程,即 T: Sync)。普通类型自动满足二者;例外:Rc(引用计数非原子)、裸指针、RefCell 等内部可变类型是 !Send!/Sync 的典型。你几乎不用写"impl Send",编译器自动推导,但报错时,错误信息就是在告诉你"哪个类型为什么不安全"。

```
use std::thread;

fn main() {
    // Rc 不是 Send:引用计数不是原子的,跨线程转移会造成计数错乱
    let rc = std::rc::Rc::new(42);
    // thread::spawn(move || println!("{}", rc)); // ERROR: Rc<i32> cannot be sent
    // 因为 Rc 的 clone 不保证原子性,另一个线程 drop 它时计数可能已错乱

    // 换成 Arc(原子引用计数)就合法—见 §9.4
    let arc = std::sync::Arc::new(42);
    thread::spawn(move || println!("Arc 可以跨线程:{}", arc)).join().unwrap();

    println!("Send/Sync 编译期防线演示完毕");
}
```

★ 重点:C 的数据竞争 vs Rust 的数据竞争

C/C++ 里两个线程无锁读写同一变量是未定义行为——可能今天正常、明天崩溃、后天在客户现场炸。Rust 里同样的意图是编译错误:

- 共享可变数据:必须放进 Arc<Mutex<T>>(或原子),编译器强制你加锁;
- 只读共享:&T 本身就允许跨线程(Sync),因为不可变借用没有竞争;
- 独占转移:move 进线程,所有权保证互斥。

结果是:数据竞争类 bug 在 Rust 程序里出现的概率从"随机的"变成"编译期不可能的"——除非你用 unsafe(第5章已述,unsafe 是唯一后门)。

§9.4 共享可变状态:Arc<Mutex<T>>、RwLock 与原子

9.4.1 Arc + Mutex:标准组合

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // Arc:原子引用计数,多线程共享所有权(对照 shared_ptr)
    // Mutex:互斥锁,提供可变访问(对照 pthread_mutex_t)
    let counter = Arc::new(Mutex::new(0u64));
    let mut handles = vec![];

    for _ in 0..8 {
        // 每线程克隆一个 Arc 句柄(计数 +1),锁本体只有一个
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            for _ in 0..1000 {
                // lock() 返回锁保护的可变引用;作用域结束自动解锁
                let mut n = c.lock().unwrap();
                *n += 1;
            }
        })));
    }

    for h in handles {
        h.join().unwrap();
    }
    // 8 线程 × 1000 次,结果必须是 8000—无竞争
    println!("最终计数:{}", *counter.lock().unwrap());
}
```

对照 C:pthread_mutex_lock/unlock 的成对出现是"忘了 unlock 就死锁"的温床;Rust 的锁是 RAII——lock() 返回的 MutexGuard 离开作用域自动解锁,编译器保证解锁发生在所有引用之后。死锁依旧可能(§9.6),但"忘了解锁"这类 bug 被类型消灭了。

9.4.2 锁中毒:panic 留下的烂摊子

```
// 若持锁线程 panic, 锁进入"中毒"状态: 后续 lock() 返回 Err
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let m = Arc::new(Mutex::new(vec![1, 2, 3]));

    let m2 = Arc::clone(&m);
    let t = thread::spawn(move || {
        let mut v = m2.lock().unwrap();
        v.push(4);
        panic!("持锁期间崩溃");           // 解锁前 panic → 锁中毒
    });
    let _ = t.join();

    // 中毒后:unwrap 会 panic; 生产做法是接管中毒状态继续用
    let v = m.lock().unwrap_or_else(|e| e.into_inner());
    println!("数据仍在:{v:?}");           // 数据完好, 只是锁标记了中毒
}
```

9.4.3 RwLock 与原子类型

```
use std::sync::{Arc, RwLock};
use std::sync::atomic::{AtomicUsize, Ordering};

fn main() {
    // 读写锁:读多写少的配置/缓存场景(对照 pthread_rwlock)
    let config = Arc::new(RwLock::new(String::from("v1")));

    // 读:可多个线程同时 hold 读锁
    let r1 = config.read().unwrap();
    println!("读锁读取:{}", *r1);
    drop(r1); // 显式释放,再拿写锁

    // 写:独占(与 Mutex 同强度,但读路径并发)
    {
        let mut w = config.write().unwrap();
        w.push_str("-hotfix");
    }

    // 原子计数:无锁,单指令完成(对照 C11 atomic_fetch_add)
    let hits = AtomicUsize::new(0);
    let mut handles = vec![];
    for _ in 0..4 {
        let h = &hits;
        handles.push(std::thread::spawn(move || {
            for _ in 0..500 {
                h.fetch_add(1, Ordering::Relaxed); // 原子自增
            }
        }));
    }
    for h in handles { h.join().unwrap(); }
    println!("原子计数:{}", hits.load(Ordering::Relaxed)); // 2000
}
```

同步原语	C 对应	Rust 类型
互斥锁	pthread_mutex_t	std::sync::Mutex(RAI1 守卫)
读写锁	pthread_rwlock_t	std::sync::RwLock
共享所有权	shared_ptr(引用计数)	Arc(原子计数)/ Rc(单线程)
原子整数	C11 atomic_int / fetch_add	AtomicUsize / AtomicI64 等
内存序	memory_order_*	Ordering::Relaxed/AcqRel/SeqCst
条件变量	pthread_cond_t	std::sync::Condvar(少用,通道常更简单)

⚠ 易错点 2:持锁期间做 IO/长时间计算

MutexGuard 的作用域要 **尽量短**:拿到值拷贝出来就 drop 锁,再做耗时工作。持锁 sleep/网络 IO/大循环 = 所有等待线程排队 = 吞吐归零,还容易诱发死锁(\$9.6)。C 程序员常犯"锁包住整个函数",Rust 的显式作用域(花括号)把"锁的存活范围"写进代码,审查时一目了然。

§9.5 通道:mpsc 生产者-消费者

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    // mpsc:multi-producer single-consumer 多生产者单消费者
    let (tx, rx) = mpsc::channel();

    // 生产者 1
    let tx1 = tx.clone(); // Sender 可以克隆(多生产者)
    thread::spawn(move || {
        for i in 1..=3 {
            tx1.send(format!("生产者1-第{i}条")).unwrap();
            thread::sleep(Duration::from_millis(50));
        }
    });

    // 生产者 2(原 tx 移入)
    thread::spawn(move || {
        for i in 1..=2 {
            tx.send(format!("生产者2-第{i}条")).unwrap();
        }
    });

    // 注意:两个 sender 都 drop 后,消费者 recv 返回 Err(通道关闭)

    // 消费者:主线程收,阻塞直到有消息或通道关闭
    while let Ok(msg) = rx.recv() {
        println!("消费:{msg}");
    }
    println!("所有生产者已关闭通道");
}
```

对照 C:这等价"线程安全队列 + 条件变量"的手写组合;mpsc 把生产者侧做成可克隆的 Sender,消费者侧 recv 阻塞、try_recv 非阻塞、全部 Sender 关闭后 recv 返回 Err——这个"优雅关闭"信号在 C 里要自己实现。send 返回 Result:接收端已关闭时 send 失败(Err),这正是"通道对方死了,别再写"的类型化表达。

§9.6 死锁:成因、识别与预防

```
// 经典死锁:两个线程互等对方持有的锁
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let a = Arc::new(Mutex::new(0u32));
    let b = Arc::new(Mutex::new(0u32));

    let a1 = Arc::clone(&a); let b1 = Arc::clone(&b);
    let t1 = thread::spawn(move || {
        let _ga = a1.lock().unwrap(); // 先拿 a
        thread::sleep(std::time::Duration::from_millis(10));
        let _gb = b1.lock().unwrap(); // 再要 b → 可能等死
    });

    let a2 = Arc::clone(&a); let b2 = Arc::clone(&b);
    let t2 = thread::spawn(move || {
        let _gb = b2.lock().unwrap(); // 先拿 b
        thread::sleep(std::time::Duration::from_millis(10));
        let _ga = a2.lock().unwrap(); // 再要 a → 与 t1 互相等待
    });

    // 上面两个线程大概率互卡(锁顺序不一致)
    t1.join().unwrap();
    t2.join().unwrap();
    println!("死锁演示(若程序挂住,说明互等发生)");
}
```

★ 重点:死锁预防三原则

- **锁顺序一致**:多把锁时,所有线程按同一全局顺序加锁(如按地址排序)——本示例的修复就是 t1/t2 都先拿 a 再拿 b;
- **持锁范围最小化**:拿锁、拷贝数据、放锁,再处理业务(§9.4 易错点 2);
- **避免持锁等待通道**:recv 阻塞在持锁状态下 = 互相等待的死锁配方;需要同步时,考虑"锁外收发,锁内只改状态"。

工具层面:lock() 返回的守卫无法跨作用域逃逸(类型系统),所以"忘了解锁"被消灭;剩余死锁主要来自"多锁顺序"与"持锁阻塞",这两类要靠设计与审查——Rust 不保证消灭,但把范围缩小到可推理的境地。

§9.7 tokio 异步入门:async/await

9.7.1 为什么需要异步

每连接一线程的模型,连接数万时线程开销(每线程约 1MB 栈 + 调度)成为瓶颈。异步编程:单个线程上跑大量"可挂起任务",IO 等待时让出线程给别的任务。C 的 epoll 事件循环 + 状态机是同一思路,但状态机要手写;Rust 的 **async/await** 让编译器生成状态机。tokio 是事实标准运行时:

```
// Cargo.toml:tokio = { version = "1", features = ["full"] }
// 异步函数:执行到 .await 时若未就绪,让出线程,稍后继续
async fn fetch_page(url: &str) -> Result<String, reqwest::Error> {
    let body = reqwest::get(url).await?.text().await?;
    Ok(body)
}

#[tokio::main]                                // 运行时入口宏
async fn main() {
    // 同时发起多个任务,共享少量线程(对照 epoll + 事件驱动)
    let urls = ["https://example.com", "https://example.org"];
    let mut handles = vec![];
    for u in urls {
        // tokio::spawn:并发执行异步任务
        handles.push(tokio::spawn(async move {
            match fetch_page(u).await {
                Ok(body) => println!("{}", u, body.len()),
                Err(e) => eprintln!("{}", e, u),
            }
        }));
    }
    for h in handles {
        let _ = h.await;                // 等待任务完成
    }
    println!("全部完成");
}
```

9.7.2 线程 vs 异步:选型准则

维度	std::thread	tokio async
并发规模	数百至数千(每线程 1MB 栈)	数万至数十万(任务为堆对象)
代码形态	同步调用,自然阅读	async/await,传染性(调用链全异步)
CPU 密集任务	天然并行(多核多线程)	需 spawn_blocking 送线程池
IO 密集任务	线程等待时占用栈与调度	等待时零占用,吞吐高
适用场景	脚本、小规模、CPU 计算	高并发服务端、网关、代理

入门建议:先线程,后异步

本章的线程知识是异步的地基:async 任务同样遵循所有权、Send/Sync(await 的任务必须是 Send 才能跨线程调度)、共享状态同样用 Arc<Mutex>。生产高并发服务直接上 tokio 生态(axum 等);脚本与小工具用 std::thread 更简单。不要“为了异步而异步”——单机小工具用线程,是工程判断力。

易错点 3:Mutex 保护"数据",不是"代码段"

持锁时间越长,竞争越大,死锁风险越高(\$9.6)。正确姿势:锁内只做“取出/放入数据”的最小操作,计算与 IO 全在锁外。对照 C 的“一把大锁包住整个函数”的坏习惯——Rust 的守卫作用域是显式花括号,把“锁的存活范围”变成可见的代码结构,审查时一眼能看出锁内有没有多余操作。

本章速查表

主题	结论 / 写法
可恢复错误	<code>fn f() -> Result<T, E>; Ok(value) / Err(e)</code>
快速传播	表达式后加 <code>? :Err</code> 则提前返回(需要返回类型是 <code>Result</code>)
不可恢复	<code>panic!("消息") / assert!(cond) / unreachable!()</code>
顶层兜底	<code>main</code> 返回 <code>Result</code> 或 <code>match</code> 后 <code>eprintln</code> + <code>process::exit</code>
创建线程	<code>thread::spawn(move {...}) -> JoinHandle</code>
等待线程	<code>handle.join() -> Result</code> (线程 <code>panic</code> 时为 <code>Err</code>)
Send / Sync	自动推导; <code>Rc</code> /裸指针/ <code>RefCell</code> 不满足; <code>Arc</code> / <code>String</code> / <code>&T</code> 满足
共享可变	<code>Arc<Mutex<T>>::clone</code> 句柄; <code>lock()</code> 得守卫,作用域结束自动解锁
锁中毒	持锁 <code>panic</code> 后 <code>lock()</code> 返回 <code>Err</code> ; <code>unwrap_or_else(e e.into_inner())</code> 接管
读写锁	<code>RwLock::read()</code> 并发读; <code>write()</code> 独占写(读多写少)
原子	<code>AtomicUsize::new / fetch_add / load(Ordering)</code>
通道	<code>mpsc::channel() -> (Sender, Receiver)</code> ; <code>Sender</code> 可 <code>clone</code> ; <code>recv</code> 阻塞
通道关闭	全部 <code>Sender drop</code> 后 <code>recv</code> 返回 <code>Err</code> ; <code>send</code> 到已关闭通道也 <code>Err</code>
死锁预防	锁顺序一致;持锁作用域最小;避免持锁 <code>recv</code> /长 <code>IO</code>
异步	<code>#[tokio::main] + async fn + .await</code> ; <code>tokio::spawn(async move {})</code>

最小必会清单

- 能说出 `Result` / `?` / `panic!` 的分工,并解释为什么它们取代了 `errno` + `abort`
- 知道生产代码 `unwrap` 的边界:只留给"逻辑上不可能失败"的调用
- 能写 `thread::spawn(move || ...) + join`,并解释 `move` 的必要性
- 能解释 `Send` 与 `Sync` 的差别(转移所有权 vs 跨线程借用),各举一个反例类型
- 能徒手写 `Arc<Mutex<T>>` 的计数器模式,并说出 `RAII` 自动解锁的意义
- 知道锁中毒是什么、怎么接管;知道 `RwLock` 与 `Mutex` 的适用场景
- 能用 `AtomicUsize` 写无锁计数,知道 `Ordering` 至少认识 `Relaxed`/`SeqCst`
- 能用 `mpsc` 写生产者-消费者,理解通道关闭信号(`recv Err`)
- 能说出死锁三大成因(锁顺序/持锁阻塞/互等)与预防手段
- 能说出线程与 `tokio` 的选型准则,并能写一个最小 `async main`

自测题

Q 1. 为什么 `errno` 不适合生产错误处理?`Rust` 的 `Result` 是怎么解决这些问题的?

 参考答案

`errno` 是全局状态:任何库函数都可能改写它,错误与返回值分离导致"查漏",多线程虽线程局部但无法携带上下文(哪个文件、哪个操作)。Result 把错误作为值随调用链显式传递:不会静默覆盖;错误携带路径/原因;? 让传播写成一行的同时保留完整信息;类型系统强制调用方处理(不处理就编译警告)。

Q 2. panic! 与 abort() 有什么本质区别?什么场景宁可 panic?

✓ 参考答案

`abort` 立即终止,不执行任何清理;`panic` 默认"栈展开",沿途执行 `Drop`(文件句柄、锁守卫、缓冲区都会被释放),可配置为 `abort(panic = "abort"` 剖面)。宁可 `panic` 的场景:前置条件被破坏且无法恢复(配置非法、状态机进入不可能分支、断言失败)——此时快速失败优于带着坏状态继续跑。生产上"外部输入错误"应走 `Result`, "内部逻辑错误"走 `panic`。

Q 3. 为什么 Rc 不能跨线程,而 Arc 可以?编译错误提示了什么?

✓ 参考答案

`Rc` 的引用计数增减不是原子操作:两个线程同时 `clone/drop` 会错乱计数,造成双重释放或提前释放(未定义行为)。`Arc` 的计数用原子指令,保证跨线程安全。编译错误 "`Rc<i32> cannot be sent between threads safely`" 直接点明类型与原因——这正是 `Send/Sync` 自动推导的价值:`C` 里同样的错误要等运行期崩溃,而且崩溃点与根源相距甚远。

Q 4. Arc<Mutex<T>> 与"全局 static mut + pthread_mutex"相比,结构上强在哪?

✓ 参考答案

三点:① 生命周期:锁与数据绑定在一个类型里,访问必须经过 `lock()`,编译器防止"绕过锁直接读写数据"(`C` 里 `static mut` 可以被直接访问,锁形同虚设);② 解锁保证:`MutexGuard RAII`,离开作用域自动解锁,不存在忘解锁的死锁;③ 所有权:`Arc` 句柄的 `clone/转移` 受所有权规则管理,不会出现悬垂指针指向已释放的锁。

Q 5. mpsc 的 recv 返回 Err 意味着什么?这个信号在 C 的手写队列里怎么表达?

✓ 参考答案

意味着所有 `Sender` 已被 `drop`(或发送失败被忽略后),不会再有任何消息到达——通道"优雅关闭"。`C` 的手写队列需要自己实现"关闭标志 + 条件变量广播",消费者醒来后还要检查标志并区分"队列空"与"队列关闭"两种状态;`mpsc` 用 `Err` 一个类型把这两种情况区分开,生产者侧 `send` 的 `Err` 则表达"接收端已消失,别再写"。

Q 6. 一个高并发 HTTP 服务,什么时候该用 tokio,什么时候用 std::thread 就够了?

✓ 参考答案

取决于并发规模与任务类型:数千并发连接、大量 `IO` 等待(网络/磁盘/下游调用)的场景,`tokio` 的"任务让出线程"带来数量级优势;`CPU` 密集或数百并发以内的服务,`std::thread` 每连接一线程简单直接。混合形态:`tokio` 服务里 `CPU` 密集段用 `spawn_blocking` 送回线程池。判断口诀:`IO` 密集 + 大规模并发 = 异步;`CPU` 密集 + 小规模 = 线程。

章末实验题:生产者-消费者任务队列

实验 9:生产者-消费者任务队列

实验目标:实现一个任务队列系统:多个生产者向通道投放任务,固定数量的 worker 线程消费并处理,共享计数器统计结果,主线程汇总。

实验要求:

- 任务 1:定义 Task 类型(编号 + 载荷),生产者线程循环投放(覆盖:线程 §9.2、通道 §9.5)
- 任务 2:固定 4 个 worker 线程,从接收端 recv 取任务并"处理"(sleep 模拟耗时)(覆盖:消费者模式 §9.5)
- 任务 3:Arc<AtomicUsize> 共享"已完成数"与"错误数"计数(覆盖:原子 §9.4)
- 任务 4:所有生产结束后,主线程 drop 自己的 Sender 使通道关闭,worker 感知 Err 退出(覆盖:通道关闭 §9.5)
- 任务 5:主线程 join 全部 worker,输出汇总(覆盖:join §9.2)
- 加分:任务内容为表达式字符串时,解析失败的任务计入错误数而不是崩溃(覆盖:Result 错误处理 §9.1)

运行方式:cargo run;预期输出 4 个 worker 各自处理的条数与总数、错误数。

第一步 **一步:**设计 Task 与结果类型,确定生产者数量、任务总数、worker 数量为常量

第二步 **二步:**建通道,生产者线程 move 各自的 Sender 克隆并投放

第三步 **三步:**写 worker:loop 中 recv,Err 时 break(通道关闭)

第四步 **四步:**加 Arc<AtomicUsize> 计数,处理每条任务后 fetch_add

第五步 **五步:**主线程 drop tx 后 join 全部 worker,打印汇总,验证多次运行计数一致

参考答案要点

```

// ===== 实验 9:生产者-消费者任务队列 =====
// 覆盖知识点:线程与 move($9.2)、mpsc 通道与优雅关闭($9.5)、
//          原子计数($9.4)、join 汇总($9.2)、Result 错误处理($9.1)

use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::mpsc::{self, Receiver, Sender};
use std::sync::Arc;
use std::thread;
use std::time::Duration;

const PRODUCERS: usize = 3;      // 生产者数量
const TASKS_PER_PRODUCER: u32 = 5; // 每生产者任务数
const WORKERS: usize = 4;        // 消费者(worker)数量

/// 任务:编号 + 载荷表达式
struct Task {
    id: u32,
    expr: String,
}

/// 处理一个任务:解析"a+b"并返回结果(失败计入错误,不崩溃)
fn process(task: &Task) -> Result<i32, String> {
    // 简单格式:"数字+数字"
    let parts: Vec<&str> = task.expr.split('+').collect();
    if parts.len() != 2 {
        return Err(format!("任务 {} 格式错误:{}", task.id, task.expr));
    }
    let a: i32 = parts[0].trim().parse()
        .map_err(|_| format!("任务 {} 左操作数非法", task.id))?;
    let b: i32 = parts[1].trim().parse()
        .map_err(|_| format!("任务 {} 右操作数非法", task.id))?;
    Ok(a + b)
}

/// 生产者:连续投放任务,随后 drop 自己的 Sender
fn producer(tx: Sender<Task>, id: usize) {
    for i in 0..TASKS_PER_PRODUCER {
        let task = Task {
            id: id as u32 * 100 + i,
            expr: format!("{}", id * 7 + i as usize, i + 1),
        };
        // send 失败 = 接收端已关闭:直接返回(优雅终止)
        if tx.send(task).is_err() {
            eprintln!("生产者 {}:通道已关闭,停止", id);
            return;
        }
    }
}

/// 消费者:循环取任务,Err 表示通道关闭,统计后退出
fn worker(rx: Receiver<Task>, done: &AtomicUsize, failed: &AtomicUsize) {
    loop {
        match rx.recv() {
            Ok(task) => {
                // 模拟处理耗时(对照真实 IO/计算)

```

```

        thread::sleep(Duration::from_millis(10));
        match process(&task) {
            Ok(result) => {
                done.fetch_add(1, Ordering::Relaxed);
                println!("任务 {}:{} = {}", task.id, task.expr, result);
            }
            Err(msg) => {
                failed.fetch_add(1, Ordering::Relaxed);
                eprintln!("{msg}");
            }
        }
    }
    Err(_) => break,    // 所有生产者已关闭:通道结束
}
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let (tx, rx) = mpsc::channel::<Task>();
    let done = Arc::new(AtomicUsize::new(0));
    let failed = Arc::new(AtomicUsize::new(0));

    // 启动 workers:每个 worker 一个接收端副本(Receiver 不可克隆,
    // 所以用 Arc<Receiver> 共享 — 生产上更常用多通道分发)
    let rx = Arc::new(rx);
    let mut worker_handles = vec![];
    for _ in 0..WORKERS {
        let rx = Arc::clone(&rx);
        let done = Arc::clone(&done);
        let failed = Arc::clone(&failed);
        worker_handles.push(thread::spawn(move || {
            worker((*rx).clone(), &done, &failed);
        }));
    }

    // 启动生产者
    let mut prod_handles = vec![];
    for id in 0..PRODUCERS {
        let tx = tx.clone();
        prod_handles.push(thread::spawn(move || producer(tx, id)));
    }
    // 主线程持有最后的 tx:立即 drop,使全部 sender 关闭
    drop(tx);
    for h in prod_handles {
        h.join().expect("生产者崩溃");
    }
    // 生产者全部退出后,所有 Sender 已 drop → worker 的 recv 返回 Err
    for h in worker_handles {
        h.join().expect("worker 崩溃");
    }

    let total = done.load(Ordering::Relaxed);
    let bad = failed.load(Ordering::Relaxed);
    println!("\n=== 汇总 ===");
    println!("成功:{total} 失败:{bad} 总任务:{}",

```



```
        PRODUCERS * TASKS_PER_PRODUCER as usize);
    if total + bad == PRODUCERS * TASKS_PER_PRODUCER as usize {
        println!("✓ 任务全部处理完毕,计数一致");
    } else {
        println!("X 计数不一致!丢失任务");
    }
    Ok(())
}
```

设计说明:三个关键点值得反复咀嚼——① `drop(tx)` 是"优雅关闭"的开关:主线程作为最后的 `Sender` 持有者主动放弃,workers 集体收到 `Err` 退出,没有任何标志位和广播;② `process` 返回 `Result`,坏任务计入 `failed` 而不是让 worker 崩溃——这是 §9.1 "可恢复错误走 `Result`"的生产形态;③ 计数用原子而非 `Mutex`,因为"自增"是单条指令——选型时先想"这个操作需要多步吗",单步就用原子。把 `done` 的类型换成 `Mutex` 再跑一遍,感受两种写法的差异,你就完成了本章的最后一课。

下一章预告:第10章 泛型与元编程——C 的宏与 `void*` 对应 Rust 的泛型系统;trait bound、关联类型、`const` 泛型、单态化,以及 `macro_rules!` 与过程宏的元编程世界。